

ĐỒNG HỒ LOGIC LAMPORT

Học phần: Ứng dụng phân tán

Quy tắc dùng chung cho toàn bộ bài tập

Mỗi tiến trình P_i có bộ đếm logic C_i , ban đầu $C_i = 0$ nếu đề bài không nói khác.

Trước mỗi sự kiện nội bộ hoặc sự kiện gửi thông điệp: $C_i \leftarrow C_i + 1$.

Khi gửi thông điệp m : gán timestamp $ts(m) = C_i$.

Khi nhận thông điệp m tại P_j : $C_j \leftarrow \max(C_j, ts(m)) + 1$. Có thể trình bày dạng cập nhật trước rồi tăng 1, miễn đảm bảo thời điểm nhận lớn hơn thời điểm gửi.

Nếu hai sự kiện không có quan hệ happens-before theo cùng tiến trình, gửi-nhận thông điệp, hoặc bắc cầu, thì chúng là tương tranh.

Bài 1. Cập nhật timestamp cho chuỗi thông điệp đơn giản

Có 3 tiến trình A, B, C. Ban đầu $C_A = C_B = C_C = 0$. Các sự kiện xảy ra theo thứ tự: A gửi M1 cho B; B nhận M1 rồi gửi M2 cho C; C nhận M2 rồi gửi M3 cho A; A nhận M3.

Yêu cầu:

- Điền timestamp của Send(M1), Send(M2), Send(M3).
- Tính giá trị đồng hồ cuối cùng của A, B, C.
- Vẽ timeline gồm 3 tiến trình và 3 thông điệp.

Gợi ý: Kê 3 trục dọc A, B, C; ghi giá trị C trước/sau mỗi sự kiện.

Bài 2. Nhận thông điệp không theo thứ tự gửi

Ban đầu $C_A = C_B = C_C = 0$. A gửi M1 cho B. C gửi M2 cho B. B nhận M2 trước, sau đó mới nhận M1.

Yêu cầu:

- Xác định timestamp gán trên M1 và M2.
- Tính giá trị CB sau khi nhận M2 và sau khi nhận M1.
- Giải thích vì sao đồng hồ logic không phản ánh thời gian vật lý tuyệt đối.

Gợi ý: Nhấn mạnh: B xử lý theo thứ tự nhận thực tế, không theo thứ tự timestamp gửi.

Bài 3. Hoàn thiện bảng sự kiện Lamport

Có 2 tiến trình P1, P2. Ban đầu $C_1 = C_2 = 0$. Trình tự: P1 có sự kiện nội bộ e11; P1 gửi m1 cho P2; P2 có sự kiện nội bộ e21; P2 nhận m1; P2 gửi m2 cho P1; P1 nhận m2; P1 có sự kiện nội bộ e12.

Yêu cầu:

- Lập bảng gồm cột: Sự kiện, Tiến trình, Loại sự kiện, Giá trị clock sau sự kiện.
- Xác định timestamp của m1 và m2.
- Chỉ ra cặp sự kiện nào có quan hệ happens-before do gửi/nhận thông điệp.

Gợi ý: Dùng bảng 4 cột để tránh nhầm giữa timestamp gửi và clock sau khi nhận.

Bài 4. So sánh hai sự kiện: xảy ra trước hay tương tranh?

Có 3 tiến trình P1, P2, P3. Các sự kiện: P1 gửi m1 cho P2; sau khi nhận m1, P2 gửi m2 cho P3; P3 có một sự kiện nội bộ x trước khi nhận m2; sau khi nhận m2, P3 có sự kiện y.

Yêu cầu:

- Xác định quan hệ giữa Send(m1) và Send(m2).
- Xác định quan hệ giữa x và Send(m1).
- Xác định quan hệ giữa Send(m1) và y.
- Giải thích cặp nào là tương tranh.

Gợi ý: Không chỉ dựa vào timestamp; cần dựa vào quan hệ cùng tiến trình, gửi-nhận, bắc cầu.

Bài 5. Tìm lỗi vi phạm nguyên lý Lamport

Một hệ thống ghi log như sau: Send(m1) tại P1 có timestamp 8, Receive(m1) tại P2 có timestamp 7; Send(m2) tại P2 có timestamp 10, Receive(m2) tại P3 có timestamp 13; Send(m3) tại P3 có timestamp 15, Receive(m3) tại P1 có timestamp 15.

Yêu cầu:

- Chỉ ra thông điệp nào vi phạm nguyên lý Lamport.
- Sửa lại timestamp nhận tối thiểu cho các trường hợp sai.
- Giải thích điều kiện receive timestamp phải lớn hơn send timestamp.

Gợi ý: Điều kiện kiểm tra nhanh: $t_{recv} > t_{send}$.

Bài 6. Tính timestamp trong kịch bản phản hồi vòng

Ban đầu $A = B = C = 0$. A gửi M1 cho B. B nhận M1, xử lý một sự kiện nội bộ, rồi gửi M2 cho C. C nhận M2, gửi ngay M3 cho A. A nhận M3, sau đó gửi M4 cho B.

Yêu cầu:

- Tính timestamp của M1, M2, M3, M4.
- Tính clock cuối cùng của A, B, C.
- Vẽ timeline và ghi timestamp tại từng mũi tên.

Gợi ý: Chú ý B có thêm một sự kiện nội bộ giữa Receive(M1) và Send(M2).